

02 Prototype

DocShield AI — working web prototype, live and open source

1. Access — deployed application and source

Both links are live and require no credentials

Application <https://trust-doc-ai.vercel.app>

Repository <https://github.com/Hajji-ahmed/TrustDoc-AI>

Upload a PDF, JPG or PNG, run the analysis, and receive an authenticity pre-check: a risk score, anomalies drawn on the document itself, and a recommendation. The interface states plainly that the result is decision support — not an expertise, not a proof.

2. Pipeline

Upload → Normalization → Multimodal analysis → Deterministic checks → Risk score → Report

01	Upload	PDF, JPG or PNG up to 10 MB. Every page of a PDF is rendered, not only the first.
02	Normalization	Pages are rasterized client-side. Preview and model input are the same image, so a flagged region always lands where the reviewer looks.
03	Multimodal analysis	All pages go to the model in a single request, which lets it compare values across pages instead of judging each page alone.
04	Deterministic checks	Code re-verifies what a model cannot be trusted with: identifier length, IBAN mod-97 key, VAT arithmetic, ordering of dates.
05	Report	Risk score out of 100, numbered anomalies, extracted fields, and a recommendation oriented toward human verification.

3. Division of labour — what the model is never asked to do

Handled by the model

- ✓ **Reads** the document and extracts fields.
- ✓ **Sees** visual manipulation: mixed fonts, broken alignment, re-inserted text.
- ✓ **Compares** pages against one another.

Never trusted to the model

- ✗ **Counting digits** — identifier length.
- ✗ **Checksums** — IBAN mod-97 key.
- ✗ **Arithmetic** — VAT and column totals.

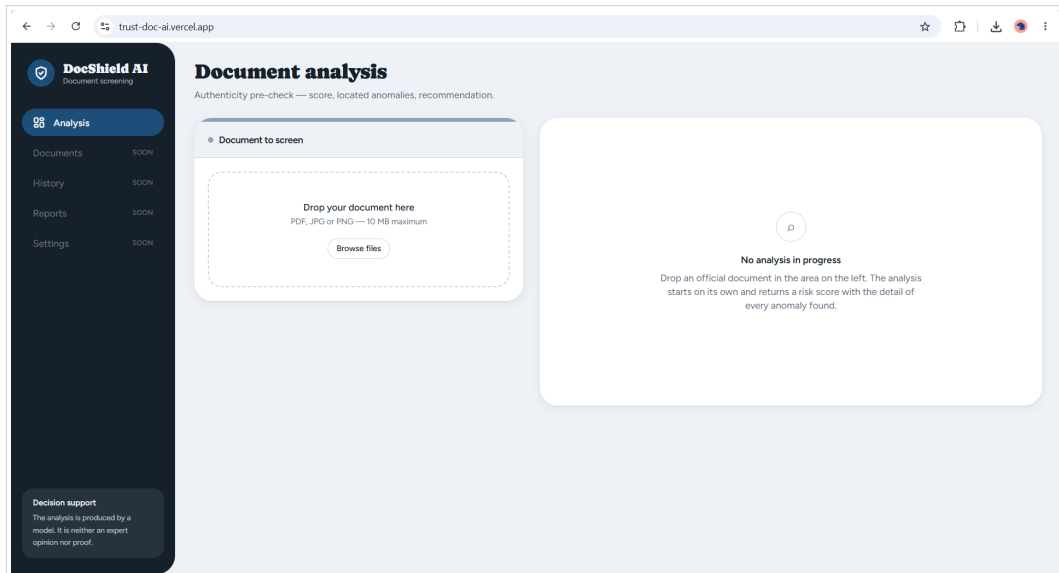
Deterministic code: exact, and auditable line by line.

A failed deterministic check raises the score floor and blocks an “accept” verdict. A model that saw nothing cannot overrule arithmetic that does not add up.

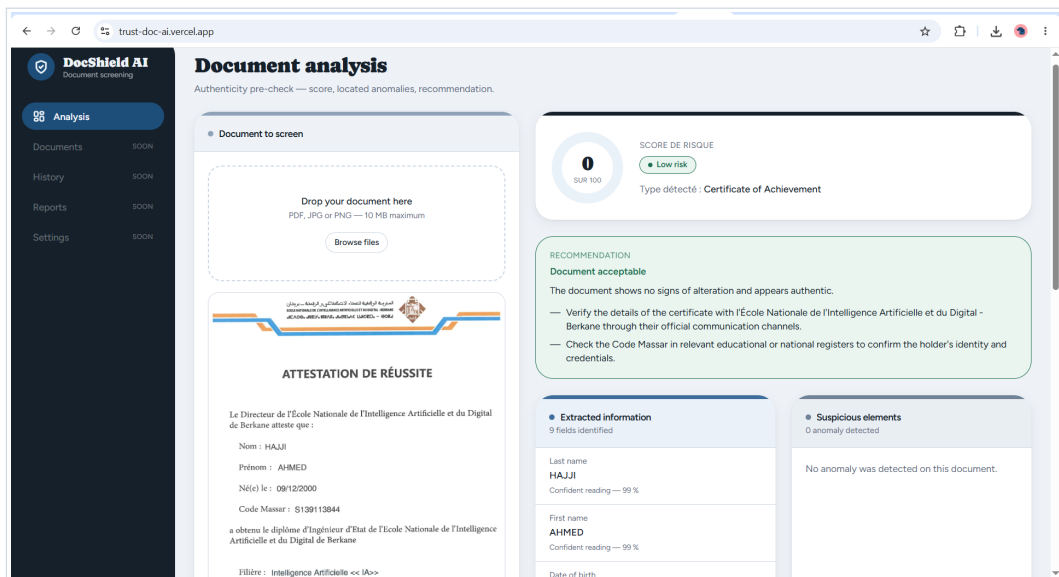
4. What the repository contains

app/api/analyze	Analysis endpoint. Validates the request, calls the model, re-validates the output, then consolidates it.
lib/prompt.ts	The system prompt: inspection method, score calibration, field-naming rules, and the ban on verifying a document against its own contact details.
lib/schema.ts	Strict JSON schema. A malformed analysis is rejected server-side, never half-displayed.
lib/validators.ts	The deterministic checks and the consolidation that keeps score, verdict and narrative from contradicting one another.
lib/pdf.ts	Multi-page rasterization, with the page cap that bounds cost per analysis.

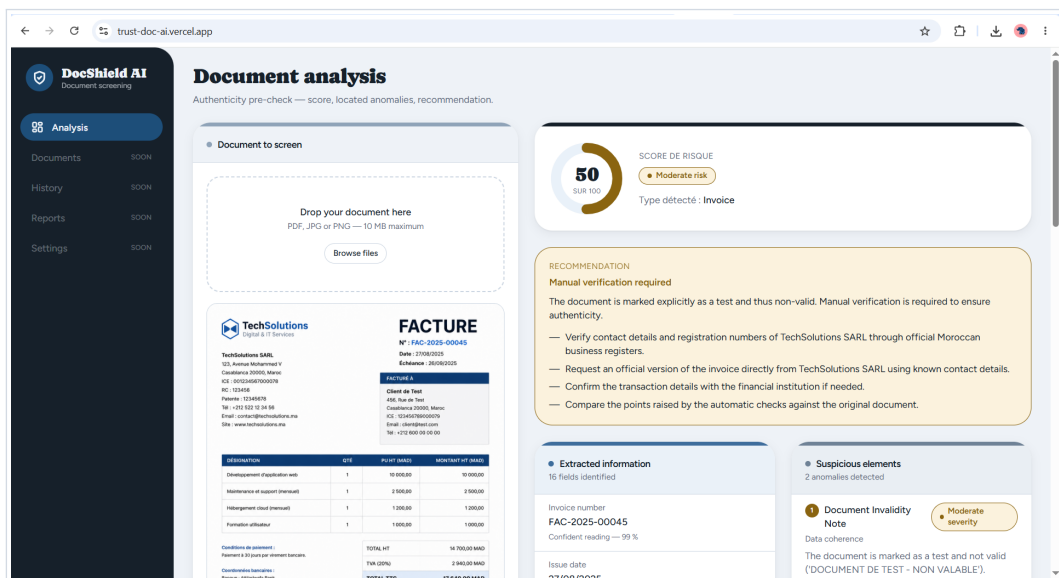
5. Screenshots



1. Initial state — the upload area, and a result panel that says what it is waiting for rather than showing an empty score.



2. A genuine certificate — 0 out of 100, no anomaly. Every point of the score is traceable to a listed finding, so nothing found means nothing scored. The next steps point to the issuing school and to public registers, never to the contact details printed on the document itself.



3. An invoice at moderate risk — two anomalies, each localised on the document and numbered against the list, with the verification steps that follow from them.

6. Positioning and honest limits

DocShield AI is a screening aid, not an authentication authority. The interface frames every output as decision support and recommends human verification when the evidence warrants it.

Out of 72h scope, part of the target architecture: OCR tuning, dedicated manipulation models, metadata forensics, cross-document checks.

Submission checklist

- ✓ Live web prototype — trust-doc-ai.vercel.app
- ✓ Public source repository — github.com/Hajji-ahmed/TrustDoc-AI
- ✓ Screenshots of the working prototype
- ✗ Optional: short demo video / screen recording